Predictive Engine Maintenance: When Should We Pull an Engine?

Case study v1.0 | November 2026

Data: NASA C-MAPSS (simulated) | Stack: Python, PostgreSQL, XGBoost

Note on framing: The business scenario is hypothetical. The data is NASA's simulated turbofan degradation data, not real airline data. All costs are illustrative assumptions. No savings are claimed, only simulated comparisons.

Note on diagrams: Figures in this version are schematic illustrations of schema and method, or explicitly marked placeholders. No model results are shown; Section 10 remains to be completed.

1 The business problem

A hypothetical airline operates a fleet of turbofan-powered aircraft. Its maintenance planners face two costly mistakes:

- **Too late:** an engine fails unexpectedly, causing cancellations, aircraft on the ground, and emergency repair costs.
- **Too early:** a healthy engine is serviced ahead of need, wasting component life and taking the aircraft out of service.

Today, many decisions follow fixed intervals. Fixed intervals treat every engine the same, even though engines degrade at different rates.

2 The decision this project supports

Primary stakeholder: Maintenance Planning Manager.

Decision: each cycle, for each engine: schedule maintenance now, or keep operating?

The model is decision support only. It does not authorize anything. Real aviation maintenance requires engineering judgment and regulatory approval.

3 Central question

Can sensor data predict an engine's Remaining Useful Life (RUL) well enough to beat fixed-interval maintenance on cost, and how robust is that answer when operating conditions change?

This splits into two tests:

1. Prediction: how accurately can RUL be estimated, and where does the model fail?
2. Decision value: does an accurate model actually lower total maintenance cost, and under which cost assumptions?

4 Approach

All splits are by engine, never by row, to prevent leakage.

Step	What	Output
1. Data	Load C-MAPSS into PostgreSQL, validate, document quality	Reproducible pipeline
2. EDA	Engine lifetimes, constant/noisy sensors, degradation trends	Sensor shortlist
3. Features	RUL labels (capped and uncapped), rolling-window features in SQL, per-condition normalization	Feature table
4. Models	Naive baseline, linear regression, XGBoost. LSTM only if justified	Model comparison
5. Generalization	Train on one subset, test on another (see below)	Failure analysis
6. Policy simulation	Fixed-interval vs RUL-threshold vs run-to-failure, with sensitivity analysis	Cost breakeven chart

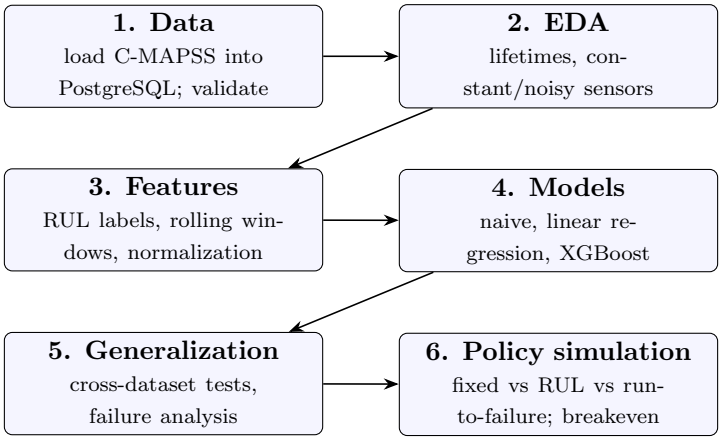


Figure 1: End-to-end pipeline. Steps 1–6 correspond to the approach table above. Schematic; implementation detail is in the repository.

Data schema (PostgreSQL)

Starter DDL (abridged):

```
CREATE TABLE engines (
  unit_id   integer PRIMARY KEY,
  dataset   text    NOT NULL,        -- FD001 .. FD004
  n_cycles  integer NOT NULL
);

CREATE TABLE sensor_readings (
  unit_id       integer NOT NULL REFERENCES engines(unit_id),
  cycle         integer NOT NULL,
  op_setting_1  real,
  op_setting_2  real,
  op_setting_3  real,
  s1 real, s2 real, /* ... */ s21 real,
  PRIMARY KEY (unit_id, cycle)
);

CREATE TABLE rul_labels (
  unit_id       integer NOT NULL REFERENCES engines(unit_id),
```

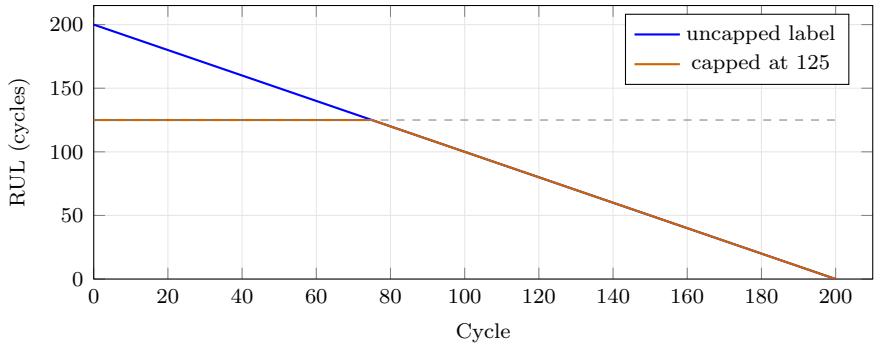


Figure 2: Schematic of RUL labelling: the uncapped label versus the 125-cycle cap tested in modeling. Illustrative only; not data.

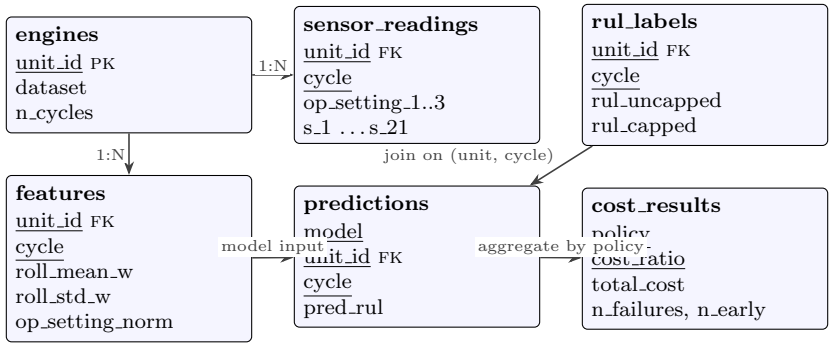


Figure 3: Abridged schema. Every `unit_id` references `engines(unit_id)`; tables join on (`unit_id`, `cycle`).

```
  cycle         integer NOT NULL,
  rul_uncapped  integer NOT NULL,
  rul_capped    integer NOT NULL,    -- min(rul_uncapped, 125)
  PRIMARY KEY (unit_id, cycle)
);

CREATE TABLE predictions (
  model     text    NOT NULL,
  unit_id   integer NOT NULL REFERENCES engines(unit_id),
  cycle     integer NOT NULL,
  pred_rul  real    NOT NULL,
  PRIMARY KEY (model, unit_id, cycle)
);
```

5 Experiments

Train	Test	Question
FD001	FD001	Baseline performance
FD001	FD002, FD004	Does it survive new operating conditions and fault modes?
FD002	FD002	In-domain comparison
FD004	FD004	In-domain comparison

Expected finding to test: models trained on FD001 will likely degrade sharply on FD002 and FD004 unless operating conditions are normalized. That outcome is a result to explain, not a failure to hide.

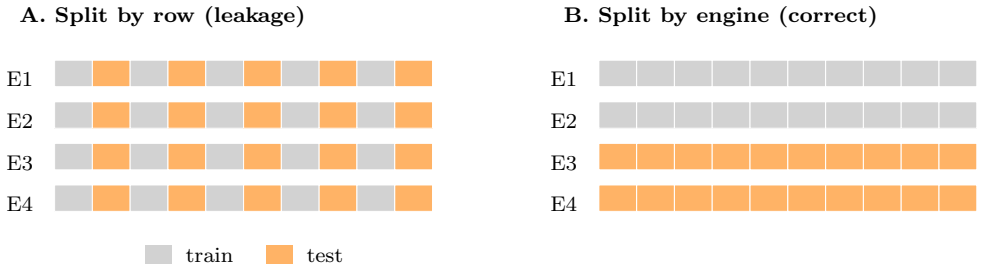


Figure 4: Train/test splitting. Splitting by row leaks information because cycles of the same engine appear in both sets; all splits in this project are by engine. Schematic.

6 Success metrics

Prediction

- RMSE and MAE
- NASA asymmetric score (late predictions penalized more than early ones)
- Overestimation rate: share of predictions where predicted RUL exceeds actual RUL
- Cross-dataset performance drop versus in-domain

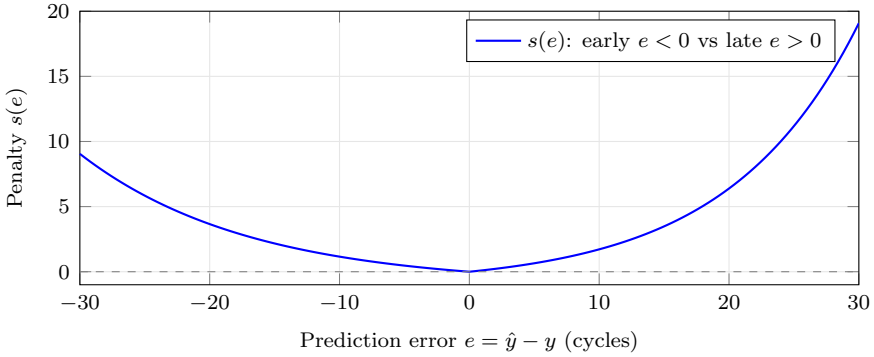


Figure 5: NASA asymmetric scoring function used in evaluation. The curve is the scoring formula itself; no model results are shown.

Decision value

- Total simulated cost per engine lifecycle
- Unplanned failures and premature interventions per policy
- Breakeven cost ratio: at what ratio of failure cost to planned-maintenance cost does RUL-based maintenance stop beating a fixed schedule?

The breakeven analysis is the core business insight. Because costs are assumed, the conclusion is a range of conditions, not a single savings number.

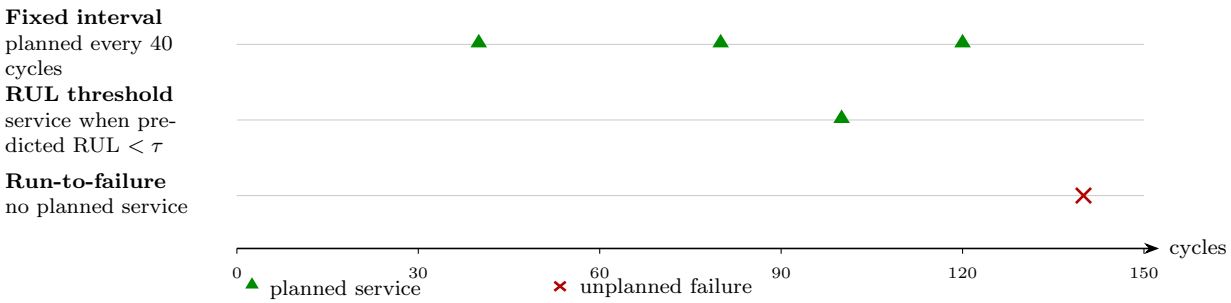


Figure 6: Schematic of the three simulated policies. Cycle positions are illustrative, not results.

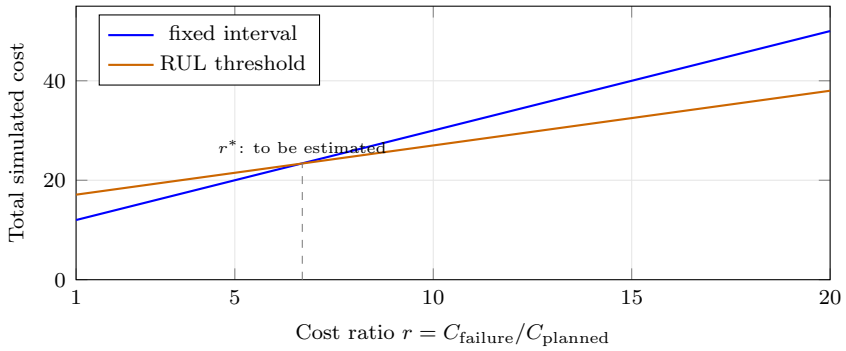


Figure 7: Concept of the breakeven chart: cost as a function of the failure-to-planned cost ratio. The shape is illustrative only; the actual curves and r^* come from the sensitivity analysis (Section 10).

7 Assumptions

1. RUL is measured in operating cycles.
2. RUL labels come from each training engine's last recorded cycle.
3. A cap near 125 cycles is a modeling choice, tested both ways.
4. Illustrative starting costs: unplanned failure = 10x the cost of planned maintenance. These are varied in sensitivity analysis.
5. Maintenance policies are simplified: one engine, one decision per cycle, no fleet-level scheduling constraints.

8 Limitations

- The data is simulated and may not reflect real engine behavior, sensor noise, or maintenance practice.
- The simulation ignores logistics such as spare availability, hangar slots, and flight schedules.
- Cost results depend on assumed values; only the sensitivity analysis is meaningful.
- RUL here is cycles to simulated failure, not a probability of an in-service event.

9 Deliverables

1. GitHub repo that runs end to end with a clear README

2. PostgreSQL schema and SQL feature queries
3. Model comparison and generalization report with error analysis
4. Cost simulation with breakeven chart
5. Short written summary of findings and recommendations
6. Simple dashboard (only after items 1–5 are done)

10 Results (to be completed)

- Best in-domain model and score: TBD
- Cross-dataset performance drop: TBD
- Where the model fails (conditions, engines, lifecycle stage): TBD
- Breakeven cost ratio: TBD
- Recommendation to the planning manager: TBD

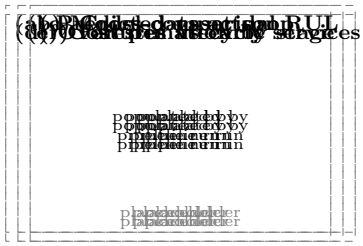


Figure 8: Placeholders for the figures referenced in Section 10. Each panel will be generated from held-out engine data and the policy simulation; no results are shown in this version.

11 Out of scope

Real-time integration, certification, accident-risk claims, claims of real airline savings, and N-CMAPSS (considered only after the main investigation is complete).